

TP CoCoMa

Planification multiagent sous incertitude

Aurélie Beynier

2025-2026

Il existe de nombreux algorithmes pour résoudre des problèmes de décision séquentielle multiagent sous incertitude.

On se propose d'expérimenter différents modèles et algorithmes du domaine. Pour ce faire, nous utiliserons « The multiagent decision process toolbox » (MADP)¹ [OSRM14].

Certains algorithmes sont également exécutables à partir du site du Thinc Lab <http://thinc.cs.uga.edu/pomdp/>. Ce site propose également les algorithmes classiques pour résoudre des POMDP.

1 Installation

1.1 "From scratch"

Téléchargez la dernière version de MADP toolbox à l'adresse suivante :
<http://www.fransoliehoek.net/fb/index.php?fuseaction=software.madp>

Décompressez l'archive.
et installez la boîte à outils avec les commandes :

```
./configure  
make  
make install
```

1.2 En salle de TME

La boîte à outils est déjà pré-installée dans le répertoire `/usr/local/madp-0.4.1` des machines de la salle 14-15 509.

Connectez-vous en ssh sur une des machines de cette salle :

```
ssh -X login@ppti-14-509-02
```

Faites varier le numéro de machine pour ne pas être tous connectés sur la même!

1. Cette boîte à outils a été développée pour Linux. Vous devez donc utiliser une machine sous Linux.

2 Prise en main

Testez la boîte à outils avec la commande suivante :

```
src/examples/example_BFS DT -h2 -v
```

Cette commande résout, à un horizon 2, le problème du tigre multiagent en utilisant un algorithme basé sur une recherche exhaustive de la solution optimale. La politique optimale est ensuite affichée.

Attention, la valeur de la politique n'est pas correcte (bug de cette version). On privilégie donc la commande suivante pour résoudre le problème de manière optimale :

```
cd src/solvers
```

```
./GMAA -G MAStar -B BFS -v -h2 DT -v
```

1. Analysez la politique obtenue.
2. Résolvez le problème à horizon 3. Que constate-t-on en ce qui concerne le temps de résolution ? Comparez la politique obtenue avec celle à horizon 2.
3. Résolvez de manière approchée le problème en utilisant la commande :

```
cd src/solvers
```

```
./GMAA -G FSPC -B AM -v -h3 DT -v
```

GMAA permet de résoudre des DEC-POMDPs. On utilise ici une méthode approchée (AM : Approximate Solution via alternating maximization) et la méthode étend uniquement le nœud le plus prometteur à chaque étape (FSPC : Forward Sweep Policy Computation).

4. Comparez avec la politique précédente.
5. Testez la résolution à horizon 5 avec la méthode exacte exhaustive et avec GMAA. Que constatez-vous ?

3 Patrouille multiagent - totalement observable

A partir du problème de patrouille sur une grille décrit en TD et formalisé sous forme de MMDP, nous nous intéressons à présent au calcul de la politique optimale pour les agents.

Ce problème est fortement inspiré du problème de la rencontre sur une grille 2x2 décrit dans [BHZ05]. Il est conseillé de récupérer cet article et de lire la description du problème qui y est faite. Une représentation de ce problème est aussi proposée dans la boîte à outils MADP. Les différences majeures entre ce problème et celui de la patrouille que nous considérons, résident dans la définition des actions et des récompenses : on ne définit pas d'action « rester sur place » et on cherche à éviter les rencontres au lieu de les favoriser.

A la ligne 159 de MMDPSolver.cpp, ajouter la ligne :

```
cout << "action policy : "<< SoftPrintVector(q);
```

recompiler et cela vous affiche la table des Q-valeurs dans le terminal

On résout le problème formalisé sous forme de MMDP avec :

```
cd src/solvers
```

```
./MMDP_Solver ../../problems/GridSmall.dpomdp --inf --discount=0.9
```

`-inf` correspond à un horizon infini. Il peut être remplacé par `-hX` où X est la valeur de l'horizon. `-discount` permet d'indiquer la valeur du facteur d'actualisation.

1. Testez la résolution du problème de rencontre sur une grille (Small Grid) à horizon 3. Quelle est la stratégie obtenue ?
2. Modifiez le fichier de description du problème de rencontre sur une grille pour formaliser le problème de patrouille dans le cas totalement observable.
3. Analysez la politique obtenue.

4 Patrouille multiagent - partiellement observable

On s'intéresse ensuite au cas partiellement observable où le problème est formalisé et résolu comme un DEC-POMDP.

1. Adaptez la formulation du problème pour traiter le cas partiellement observable.
2. Résolvez le problème en utilisant GMAA. Faites varier les options de résolution ainsi que l'horizon. Analysez les résultats.

5 Patrouille multiagent sur un graphe

On souhaite maintenant permettre de considérer des environnements plus généraux représentés par des graphes. Les nœuds représenteront les positions possibles et les liens entre les nœuds représenteront les déplacements possibles. On supposera dans cette partie qu'un déplacement entre deux nœuds directement connectés est réalisée avec succès dans 80% des cas et échoue le reste du tant. Dans ce dernier cas, l'agent reste sur place.

1. Proposez une formulation du problème pour un graphe correspondant à une ligne de 4 nœuds puis à un cercle.
2. Résolvez le problème et analysez les résultats.

Références

- [ASY⁺12] B. An, E. Shieh, R. Yang, M. Tambe, C. Baldwin, J. DiRenzo, B. Maule, and G. Meyer, *Protect : a deployed game theoretic system*

to protect the ports of the united states, Proceedings of the 11th International Conference on Autonomous Agents and Multiagent Systems - Volume 1, AAMAS '12, 2012, pp. 13–20.

- [BHZ05] D. Bernstein, E.A Hansen, and S. Zilberstein, *Bounded policy iteration for decentralized pomdps*, Proceedings of the Nineteenth International Joint Conference on Artificial Intelligence (Edinburgh, Scotland), 2005.
- [Che04] Y. Chevaleyre, *Theoretical analysis of the multi-agent patrolling problem*, Proceedings of the IEEE/WIC/ACM International Conference on Intelligent Agent Technology (Washington, DC, USA), IAT '04, IEEE Computer Society, 2004, pp. 302–308.
- [OSRM14] F. Oliehoek, M. Spaan, P. Robbel, and J. Messias, *The madp toolbox 0.3*, Tech. report, 2014.
- [SAK14] E. Sless, N. Agmon, and S. Kraus, *Multi-robot adversarial patrolling : facing coordinated attacks*, International joint Conference on Autonomous Agents and Multi Agent Systems (AAMAS), 2014.
- [SJY⁺14] Eric Shieh, Albert Xin Jiang, Amulya Yadav, Pradeep Varakantham, and Milind Tambe, *Unleashing dec-mdps in security games : Enabling effective defender teamwork*, European Conference on Artificial Intelligence (ECAI), 2014.